Bordignon Giacomo 897270

REPORT PROGETTO TAW

ARCHITETTURA DEL SISTEMA:

L'architettura del sistema prevede l'utilizzo di 3 principali componenti:

Database	Backend	Frontend
MongoDB	Express - Node.js	Angular

Il database management system (DBMS) utilizzato per il progetto è stato MongoDB, si tratta di un database non relazionale orientato ai documenti. Al suo interno sono presenti una serie di collezioni contenenti dei documenti che rappresentano i dati principali utilizzato dall'applicazione.

Il backend è statto sviluppato con Express, un modulo di Node.js che fornisce una serie di middleware per la costruzione di applicazioni web, tra i più interessanti troviamo middleware per il parsing dei file JSON, gestione delle sessioni, compressione dei dati e gestione degli errori che possono accadere durante l'esecuzione. Consente inoltre di risolvere problemi come la gestione delle rotte tra i vari endpoint dell'API e attraverso la libreria Mongoose è in grado di poter interagire con il database MongoDB.

Esso utilizza anche uno speciale middleware chiamato cors() che consente di poter comunicare con diversi container appartenenti alla stessa rete in Docker. L'uso di Docker serve per non avere problemi di compatibilità tra le versioni delle varie dipendenze.

Il frontend invece utilizza come framework Angular, molto utilizzato nello sviluppo di applicazioni SPA lato client. È un framework scritto con il linguaggio Typescript e utilizza il modello Model-View-Controller MVC che consente di separare la logica dalla presentazione delle informazioni.

Consente di poter visualizzare attraverso l'utilizzo di un browser le informazioni fornite dal backend sottoforma di file in formato JSON.

Esso è suddiviso in services e components, in cui i *services* comunicano con il backend attraverso delle richieste, mentre i *components* si occupano di visualizzare le informazioni provenienti dai *services*.

DATA MODEL:

Le collections create per lo sviluppo di questa web app sono le seguenti:

· Users

Contengono tutti gli utenti, compresi compagnie aeree e amministratore.

I campi presenti in ogni suo documento sono:

1. _id : ObjectId (univoco)
2. Mail : string (univoca)
3. Username : string
4. Password : string
5. passportID: string
6. roles : string
7. tickets : Tickets [ ]

Bordignon Giacomo 897270

· Airlines

Si riferiscono esclusivamente alle compagnie aeree.

Contengono:

1. _signature : string (univoca) corrisponde alla sigla della compagnia aerea
2. Name : string
3. Revenue : number
Indica il fatturato generato dalla vendita dei biglietti.
4. Mail : string usata come collegamento con gli utenti
5. mustChangePassword : boolean indica se è necessario cambiare la password
6. aircrafts : Aircrafts [ ]

· Aircrafts

Contengono tutti gli aerei delle varie compagnie.

I suoi campi sono:

1. _numAircraft : string (univoco)
2. Seats: Seats[ ]
3. Modello : string

· Seats

Contengono le caratteristiche dei posti per ciascun aereo

I suoi campi sono:

1. _numSeat : string (univoco)
2. Class : string
3. Legroom : boolean
Indica la presenza dello spazio per le gambe maggiorato (prevalentemente presenti nelle classi Business e First).
4. isFree : boolean
Indica se il posto è libero oppure no.

· Airports

Contengono tutti gli aeroporti.

I suoi campi sono:

1. _code : string (univoco) codice per identificare l'aeroporto
2. Name : string Sigla dell'aeroporto secondo lo standard IATA
3. City : string Città a cui appartiene l'aeroporto

Bordignon Giacomo 897270

· Tickets

Contiene tutti i biglietti emessi dalle compagnie aeree e acquistati dai passeggeri

I suoi campi sono:

1. _code : string (univoco)
2. User : ObjectId
   Viene usato come collegamento con l'utente; si rivelerà utile al momento dell'acquisto del biglietto.
3. Seat : string
   Indica il posto assegnato a quel biglietto
4. Flight : string
   Indica il codice del volo a cui è assegnato il biglietto
5. Price : number
   Indica il prezzo del biglietto
6. numBag : number
   Indica il numero dei bagagli

· Routes

Contengono tutte le rotte previste per i vari voli.

I suoi campi sono:

1. _id : ObjectId (univoco)
   Identifica la rotta specifica
2. Destination : string
   Indica la destinazione
3. Departure : string
   Indica la partenza
4. LayoverFuel : string [ ]
   Indica l'eventuale fermata intermedia

· Flights

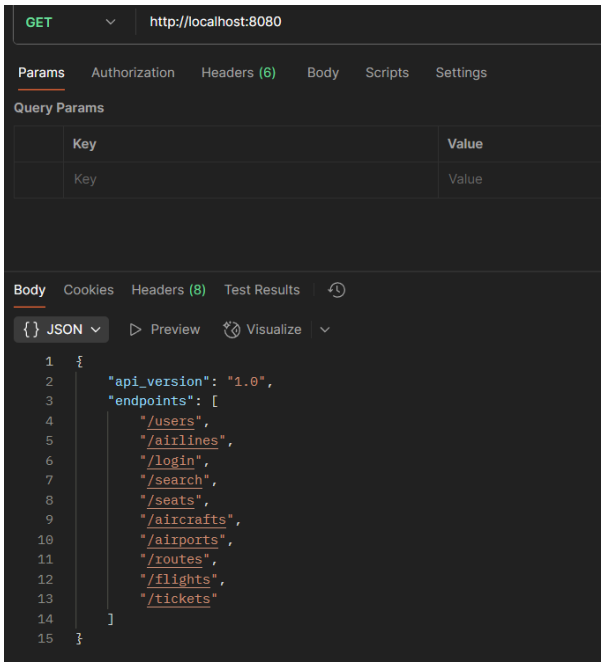
Contengono tutti i voli decisi dalle varie compagnie aeree

I suoi campi sono:

1. _flightCode : string (univoco)
   Indica il codice del volo, solitamente identificati con la signature della compagnia aerea e una sequenza di uno o quattro numeri.
2. dataOra : Date
   Indica la data e l'ora di partenza del volo secondo lo standard ...
3. route : ObjectId
   Indica il riferimento alla rotta seguita dal volo
4. aircraft : string
   Indica l'identificativo dell'aereo che effettuerà il volo
5. duration : number
   Indica la durata del volo in minuti

REST API:

L'API utilizza i seguenti endpoints preceduti da **/api/v1** per indicare che si tratta di un'API e la relativa versione:

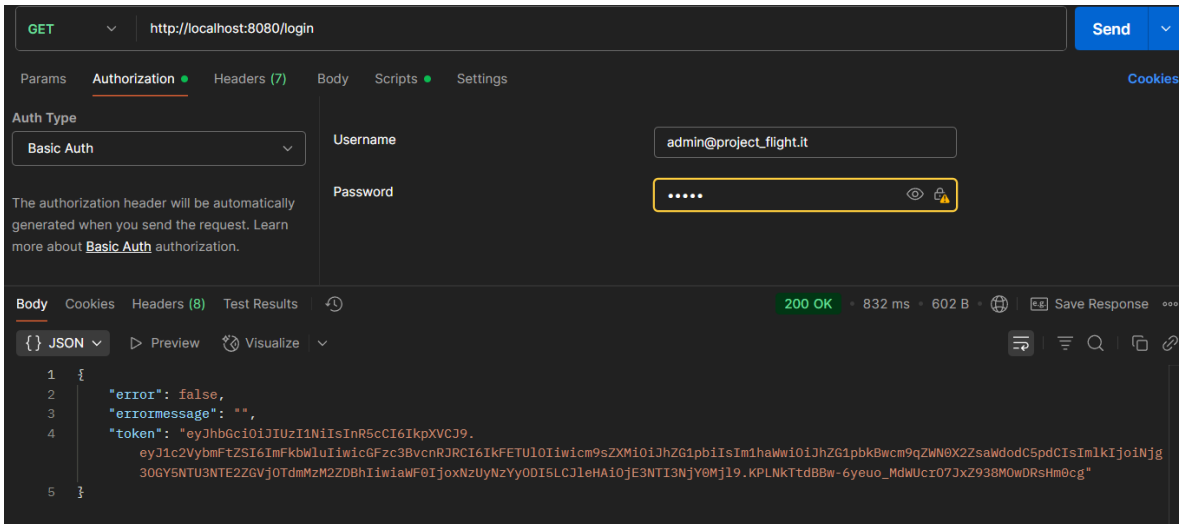


· **/login**

È una richiesta di tipo GET necessaria per l'autenticazione dei vari utenti. I parametri passati sono la mail e la password.

Esso utilizza un'autenticazione di tipo Basic, presente nella dipendenza Passport, in cui vengono controllate le credenziali e nel caso in cui siano corrette vengono codificati dei dati all'interno di una stringa detta JSON Web Token.

Questo token verrà infine firmato e inviato all'utente per poter essere utilizzato nelle successive richieste dove servirà l'autenticazione, in questo caso prende il nome di Bearer Token.



· **/users**

- Se è una richiesta di tipo POST utilizzata per la registrazione di un nuovo utente, viene impedita lato frontend se si è un airline.

Se si tratta di un utente con ruolo passenger i parametri di input saranno quelli di users, ossia mail, username, password, passportID, ruolo;

- Se è una richiesta di tipo GET non ha bisogno di parametri, ritorna tutti gli utenti il cui ruolo è passenger e può essere eseguita solo se si è un admin. Per effettuare la ricerca viene utilizzata la mail che è univoca.

· **/users/:mail**

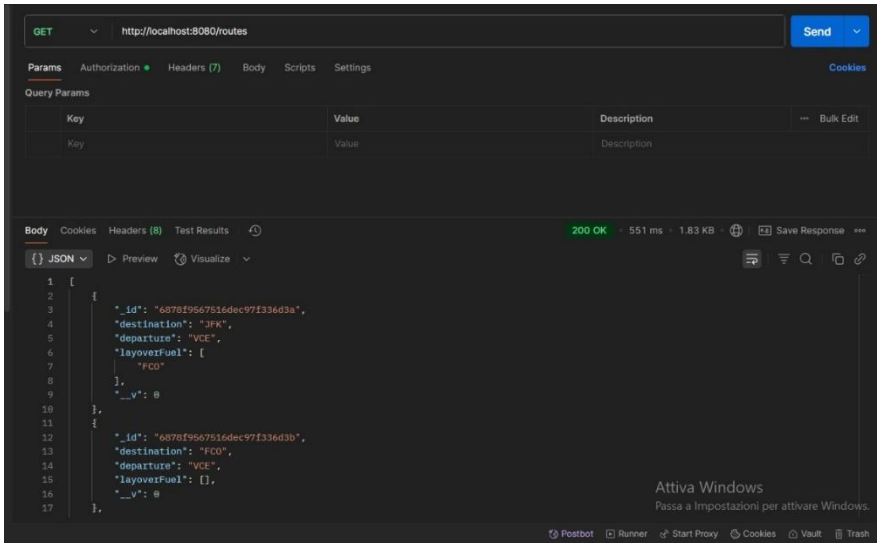
È una richiesta di tipo DELETE necessaria per l'eliminazione di un utente. Viene eseguita passando come parametro la mail dell'utente da eliminare e può essere eseguita solamente dall'admin.

· **/users/:id**

È una richiesta di tipo PUT e serve per modificare i dati dell'utente. I parametri utilizzati in input sono l'id dell'utente e i parametri che si intende modificare che possono essere password, username, mail, passportID.

· **/routes**

- Se è una richiesta di tipo GET ritorna tutte le rotte presenti nel database. Non prende nessun parametro.



- Se è una richiesta di tipo POST serve per aggiungere una nuova rotta. Essa può essere eseguita solo se è stato effettuato il login da parte di un utente con il ruolo di airline. I parametri inseriti all'interno del body sono: la partenza, la destinazione e l'eventuale scalo.

· **/routes/:id**

- Se è una richiesta di tipo PUT serve per la modifica dei dati delle rotte, in particolare è possibile modificare la partenza, la destinazione e l'eventuale scalo. Tutti e tre i dati sono riferiti alle sigle degli aeroporti.

- Se è una richiesta di tipo DELETE serve per la cancellazione delle rotte, può essere effettuata solo dalle compagnie aeree e prende come parametro l'id della rotta da cancellare.

· **/airlines:/mail**

È una richiesta di tipo GET serve per ottenere le informazioni sulla compagnia aerea corrispondente alla mail passata come parametro. Può essere eseguita solo dalla compagnia aerea.

· **/airlines**

- Se è una richiesta di tipo GET, eseguibile solo dall'admin e ritorna le informazioni di tutte le compagnie aeree presenti nel database. Non necessita di parametri in input.

- Se è una richiesta di tipo POST è eseguibile solo dall'admin e inserisce una nuova compagnia aerea. I parametri necessari sono _signature, nome, mail e una password temporanea, che verrà cambiata al primo login.

· **/airlines/:mail/aircrafts**

È una richiesta di tipo GET, eseguibile solo dalla compagnia aerea e restituisce tutti i velivoli appartenenti alla compagnia associata alla mail passata come parametro.

· **/airlines/:id**

- Se è una richiesta di tipo PUT serve per modificare alcune informazioni della compagnia aerea, quali nome, revenue e gli aerei. Può essere eseguita da un utente il cui ruolo è airline.

- Se è una richiesta di tipo DELETE serve per rimuovere una compagnia aerea. Può essere eseguita solo dall'admin e il parametro necessario è la _signature della compagnia aerea.

· **/airlines/:id/setup**

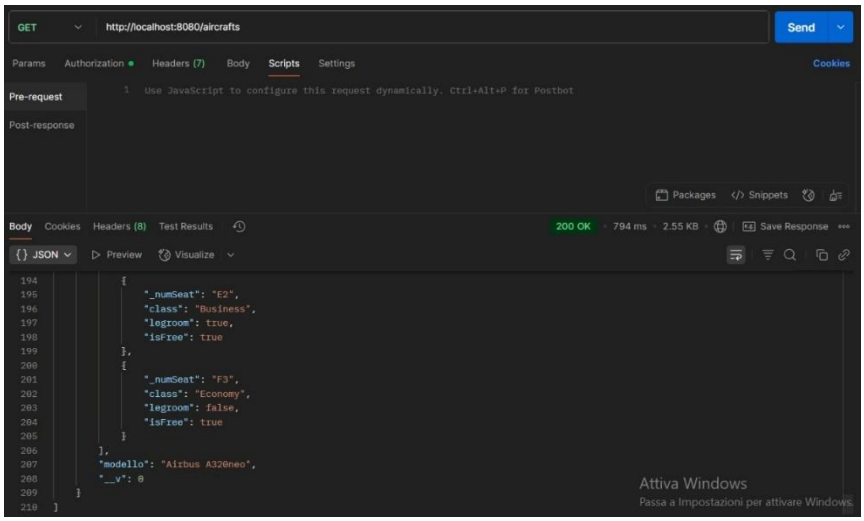
È una richiesta di tipo PUT e serve per il cambio password al momento del primo login da parte delle compagnie aeree.

Tra i parametri in input è necessaria almeno la nuova password.

· **/aircrafts**

- Se è una richiesta di tipo POST serve per aggiungere nuovi aerei, può essere eseguita solo dalla compagnia aerea e prende come parametri in input il numero dell'aereo, un array di posti e il modello.

- Se è una richiesta di tipo GET ritorna tutti gli aerei presenti nel database. Non necessita di nessun parametro in input.



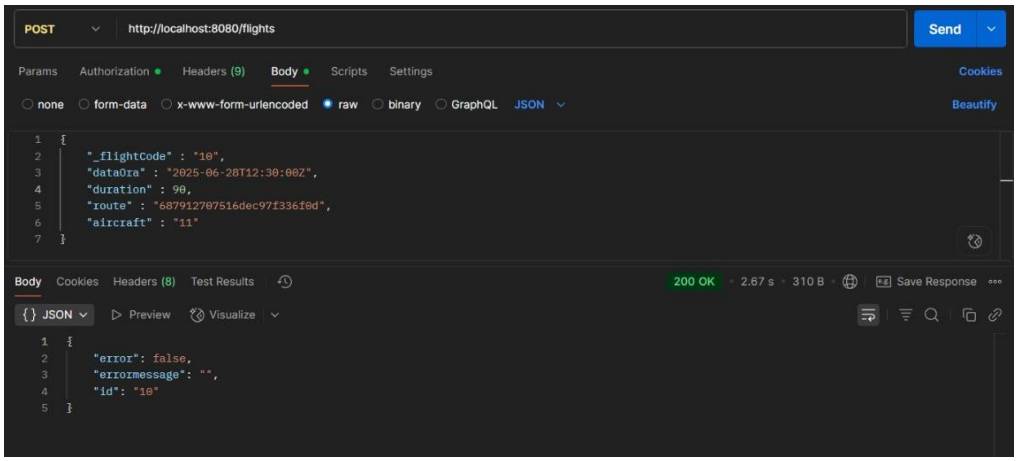
· **/aircrafts/:id**

- Se una richiesta di tipo PUT serve per modificare alcuni dati riguardanti l'aereo come i posti presenti, il modello e il numero dell'aereo da modificare. Può essere eseguita solo dalla compagnia aerea.

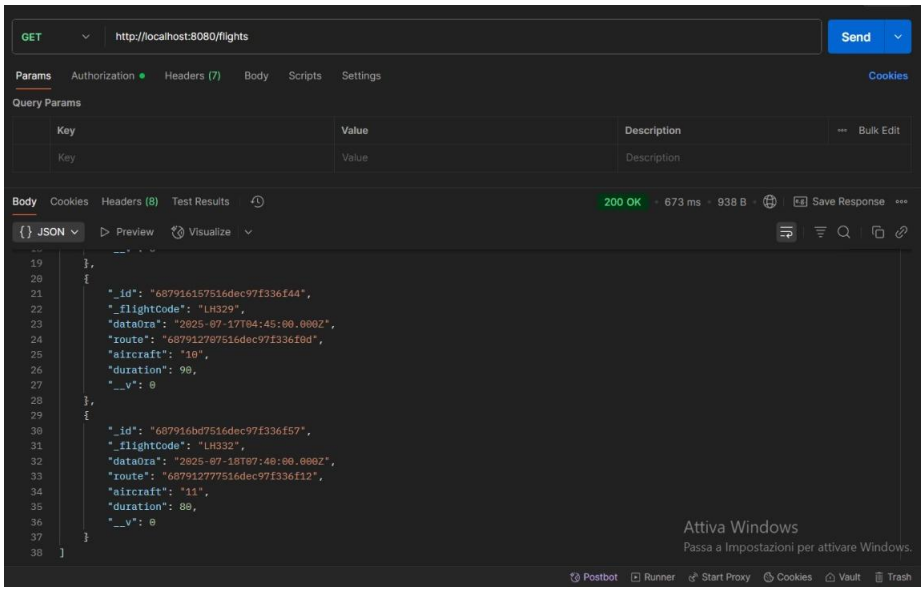
- Se è una richiesta di tipo DELETE serve per rimuovere un aereo, necessita come unico parametro in input il numero dell'aereo da cancellare e può essere eseguita anch'essa solo dalla compagnia aerea.

· **/flights**

- Se è una richiesta POST serve per aggiungere un nuovo volo per una rotta già esistente, può essere eseguito solo da un utente con ruolo airline. I parametri necessari sono: _flightCode è un codice che identifica il singolo volo, dataOra indica la data e l'ora di partenza del volo, aircraft indica il numero dell'aereo assegnato a quel volo e la durata in minuti.



- Se è una richiesta di tipo GET ritorna i voli eseguiti dagli aerei di tale compagnia. Non necessita di parametri in input.



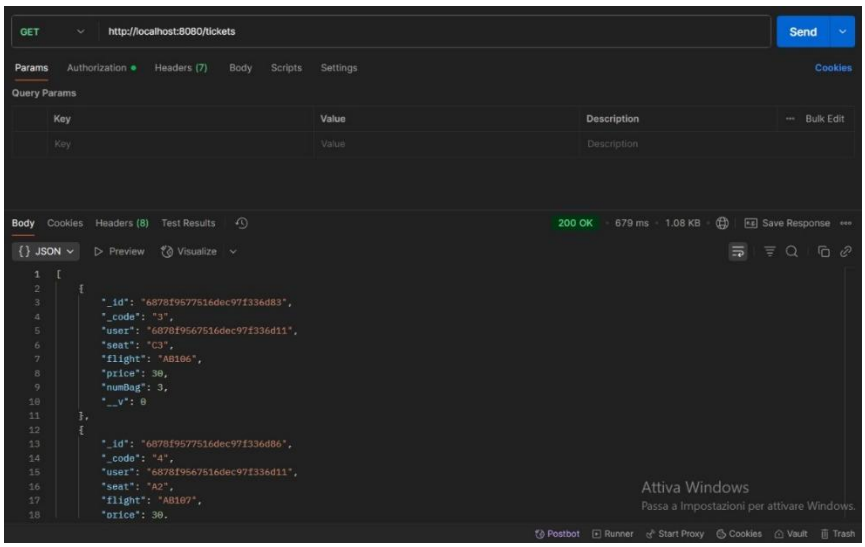
· **/flights/:id**

- Se è una richiesta di tipo PUT serve per modificare alcune informazioni riguardanti i voli già creati, quali dataOra, gli aircrafts, la rotta e la durata del volo. Può essere eseguita solo dalla compagnia aerea.

- Se è una richiesta di tipo DELETE serve per cancellare un volo già inserito. Può essere eseguita solo dalla compagnia aerea e l'unico parametro in input è il codice del volo da cancellare.

· **/tickets**

- Se è una richiesta di tipo GET ritorna tutti i biglietti associati all'utente che ha fatto il login. Può essere eseguita dai passeggeri e dalle compagnie aeree.



- Se è una richiesta di tipo POST può essere eseguita da qualsiasi utente con ruolo passeggero o compagnia aerea.

Se si tratta del primo si procede con la rimozione del biglietto acquistato dall'array dei tickets presente nella compagnia aerea e viene aggiunto a quello dell'utente che lo ha acquistato. Inoltre il posto assegnato al biglietto viene marcato come occupato.

Viene utilizzata solo dopo aver effettuato la ricerca e il login.

Se si tratta, invece, del secondo il biglietto viene aggiunto alla collection dei tickets e nell'array dei tickets della compagnia.

I parametri di input necessari sono: _code che indica il codice del biglietto, user indica l'utente a cui è assegnato, seat indica il posto, flight indica il volo, price indica il prezzo del biglietto e numBag indica il numero dei bagagli ammessi.

· **/tickets/:id**

- Se è una richiesta di tipo PUT serve per modificare alcuni dati del biglietto. Può essere eseguita solo dalla compagnia aerea. I parametri di input sono il codice del biglietto da cui si possono ricavare le informazioni di seat, numBag e price che si possono modificare in caso di necessità.

- Se è una richiesta di tipo DELETE può essere eseguita dalla compagnia aerea e dal passeggero.

Nel primo caso si tratta della cancellazione vera e propria del biglietto passando come input il codice del biglietto da rimuovere.

Nel secondo caso, invece, il biglietto non è più associato al passeggero e ritorna alla compagnia. Il posto assegnato torna ad essere libero.

Bordignon Giacomo 897270

· **/airports**

- Se è una richiesta GET vengono ritornati tutti gli aeroporti presenti nel database. Non necessita di alcun parametro in input. Viene eseguita solo dall'admin.

- Se è una richiesta POST può essere eseguita solo dall'admin e inserisce un nuovo aeroporto nel database. I parametri di input sono: _code è un codice che identifica l'aeroporto, name che corrisponde alla sigla dell'aeroporto e la città.

· **/airports/:id**

- Se è una richiesta di tipo PUT è eseguibile solo dall'admin e serve per modificare la città e/o il nome dell'aeroporto. Il parametro necessario è: il codice dell'aeroporto.

- Se è una richiesta di tipo DELETE è eseguibile solo dall'admin e serve per cancellare un aeroporto. Il parametro necessario è il codice dell'aeroporto da eliminare.

· **/seats**

- Se è una richiesta di tipo GET restituisce tutti i posti assegnati ai vari aerei. Non necessita di nessun parametro in input.

- Se è una richiesta di tipo POST serve per creare nuovi posti, può essere eseguita solo dalla compagnia aerea. I parametri necessari sono: _numSeat indica il numero del posto, class indica la classe, legroom indica se è previsto spazio per le gambe e isFree che indica se il posto è libero.

· **/seats/:id**

- Se una richiesta di tipo PUT può essere eseguita dalla compagnia aerea, serve per modificare alcune informazioni sul posto passato in input. Altri parametri di input possono essere la classe, legroom e isFree.

· **/search**

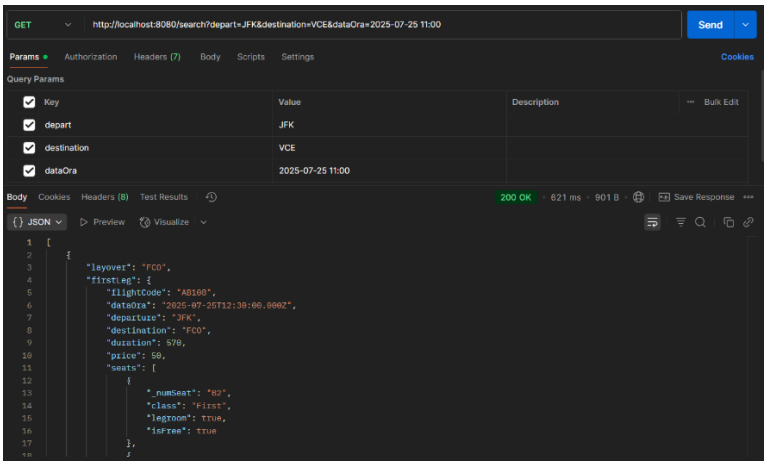
È una richiesta di tipo GET, può essere eseguita da chiunque, infatti non prevede l'autenticazione.

I parametri di input sono: la partenza, la destinazione, data e ora di partenza.

In caso di voli diretti restituisce: il codice del biglietto, la data e ora di partenza, partenza, destinazione, durata, il prezzo in base alla classe di ciascun posto libero e i posti liberi.

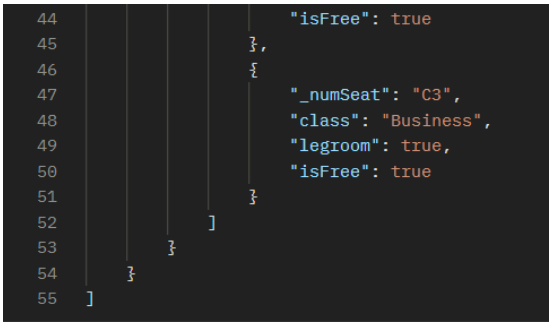
In caso di voli con scalo restituisce il biglietto per il volo dalla partenza fino allo scalo e quello per il volo dallo scalo fino alla destinazione.

I biglietti restituiti in output sono in ordine crescente rispetto al numero di posto assegnato al singolo biglietto.





```
            {
                "_numSeat": "B3",
                "class": "Economy",
                "legroom": false,
                "isFree": true
            }
        ]
    },
    "secondLeg": {
        "flightCode": "AB109",
        "dataOra": "2025-07-26T00:00:00.000Z",
        "departure": "FCO",
        "destination": "VCE",
        "duration": 70,
        "price": 50,
        "seats": [
            {
                "_numSeat": "C1",
                "class": "First",
                "legroom": true,
                "isFree": true
            },
            {
                "_numSeat": "C2",
                "class": "Economy",
                "legroom": true,
                "isFree": true
```



```
                "isFree": true
            },
            {
                "_numSeat": "C3",
                "class": "Business",
                "legroom": true,
                "isFree": true
            }
        ]
    }
]
```

DESCRIZIONE FRONTEND:

Il frontend è stato sviluppato utilizzando il framework Angular ed è stato suddiviso tra components e services.

Per ogni collection presente in MongoDB è stato creato un service che si occupa delle varie chiamate all'API e un component per poter visualizzare le varie informazioni.

· Change-password: component che si occupa della pagina del cambio password per il primo login delle compagnie aeree

· Admin: component e service per interagire con le operazioni eseguibili dall'admin

· User-login: component per la gestione del login

· User-signup: component per la gestione della registrazione

· Passengers: component e service che permette di interagire con tutte le funzionalità attribuite ai passeggeri

· Dashboard: component che in base al ruolo dell'utente che esegue il login mostra una pagina differente

· Aircrafts: service per gestire le richieste con il backend riguardanti gli aircrafts, mentre il componente è integrato all'interno di airlines.

· Airlines: component e service che permettono di interagire con le funzionalità previste per le compagnie aeree.

· Airports: service che permette di interagire con le richieste degli aeroporti, il component è integrato all'interno di admin

· Flights: component e service per la gestione dei voli

· Routes: component e service per la gestione delle rotte per i voli

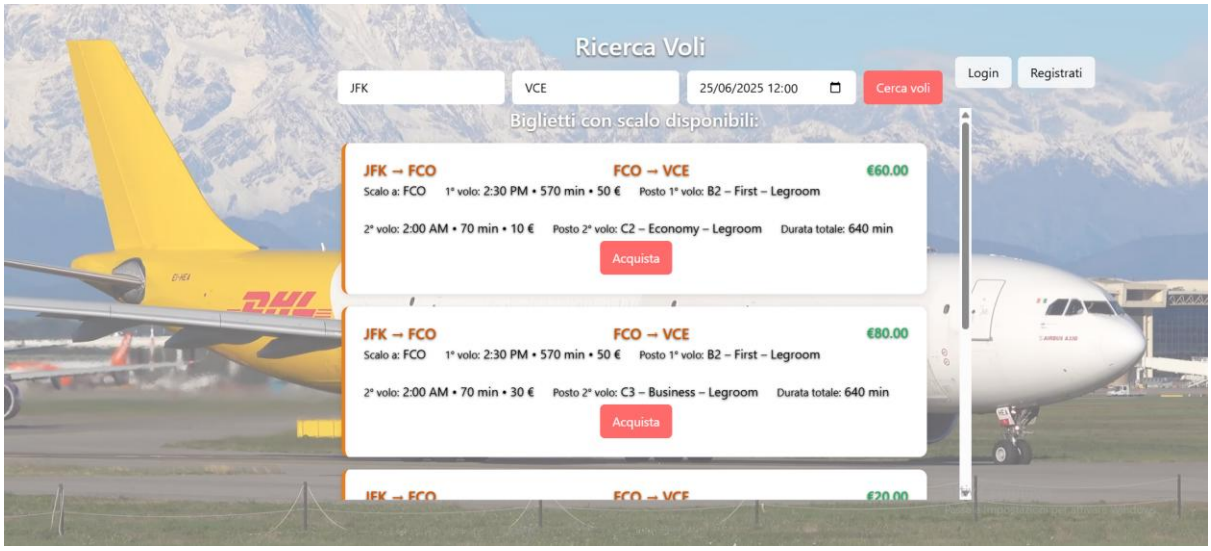
· Search: component e service per la ricerca dei biglietti e per procedere all'acquisto

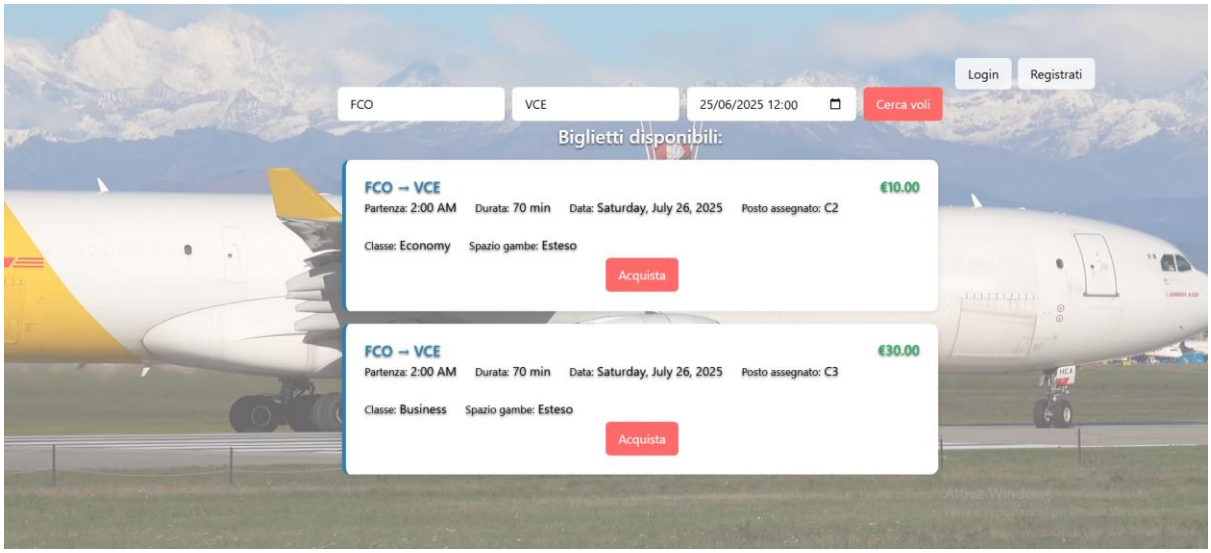
· Tickets: service per l'invio delle richeste al backend riguardanti i ticket, ma il component è integrato in airlines e passengers

Le routes previste in Angular sono:

· **/search** -> Per eseguire la ricerca. Inoltre sono previsti 2 pulsanti per login e registrazioni che portano alle relative pagine e il pulsante acquista che riconduce al login, per l'acquisto effettivo del biglietto selezionato.

Bordignon Giacomo 897270





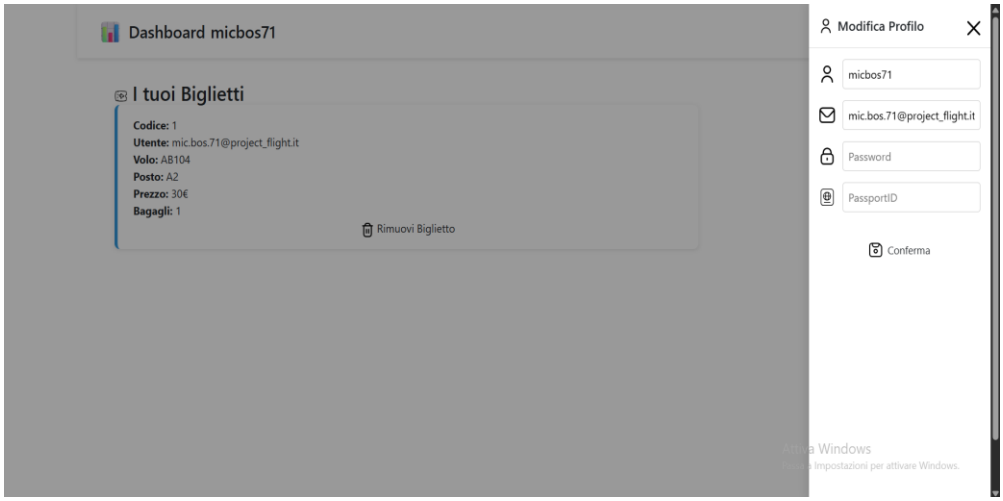
· **/login** -> Serve per effettuare il login. Se si è una compagnia aerea ed è il primo login allora si viene indirizzati alla pagina di cambio password.

· **/signup** -> Una volta registrati si viene indirizzati alla pagina di login

· **/change-password** -> Una volta cliccato il pulsante cambia password si viene reindirizzati alla propria dashboard.

· **/dashboard** -> Una volta terminato il login, si presenta la dashboard all'interno della quale sono presenti le principali funzioni che un determinato utente può sfruttare a seconda del proprio ruolo.

o Se l'utente è passenger (passeggero) allora può solo visualizzare ed eliminare i biglietti acquistati e modificare i dati inseriti al momento della registrazione.



- Se l'utente è airline (compagnia aerea) allora può:

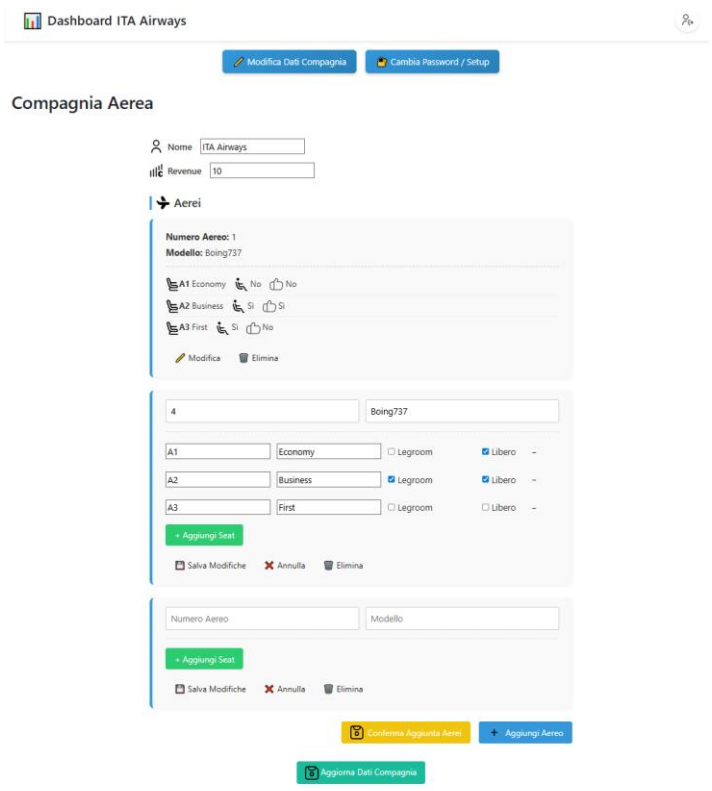
§ Modificare dati riguardanti il nome della compagnia e la revenue (il fatturato)

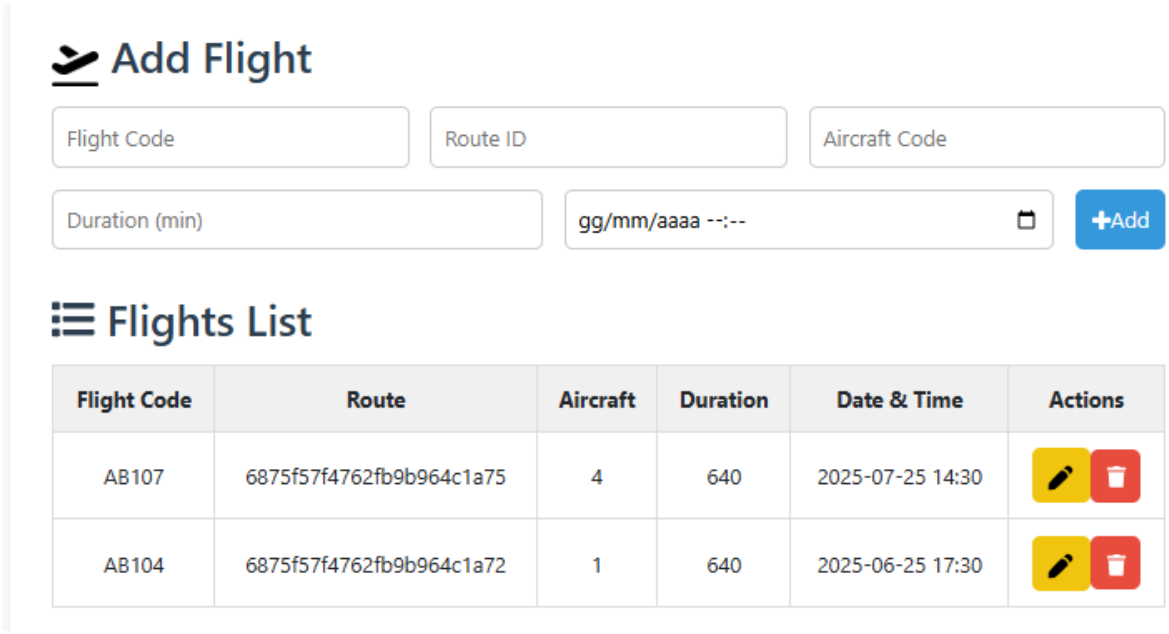
§ Aggiungere, modificare ed eliminare velivoli;

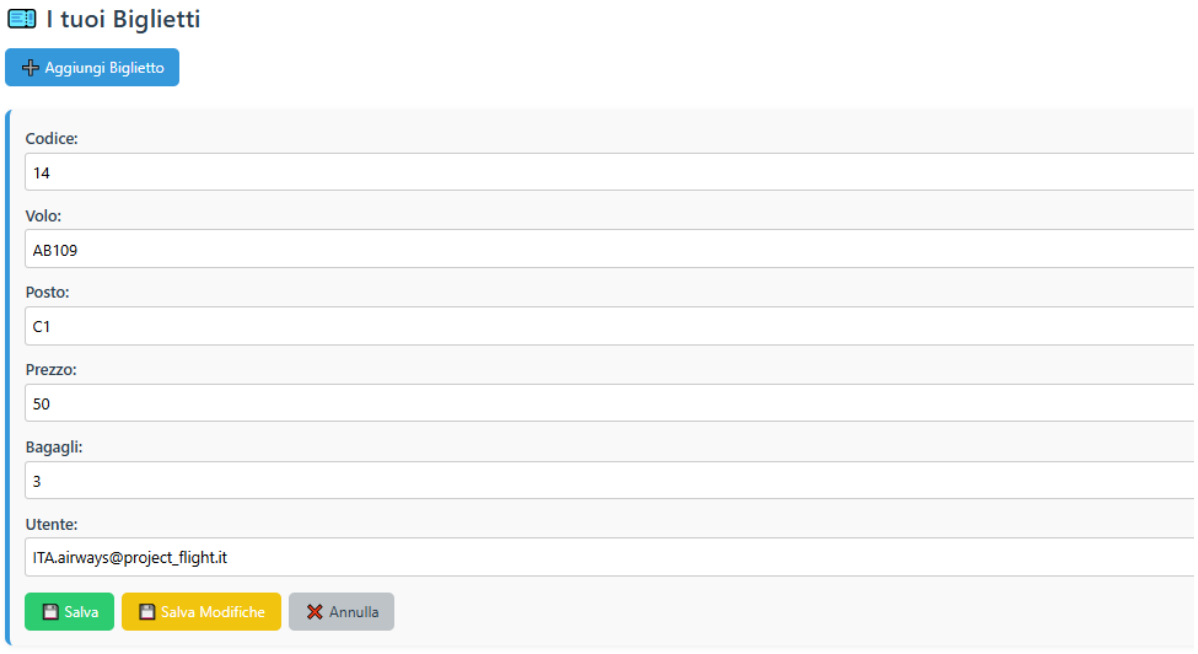
§ Aggiungere, modificare ed eliminare voli;

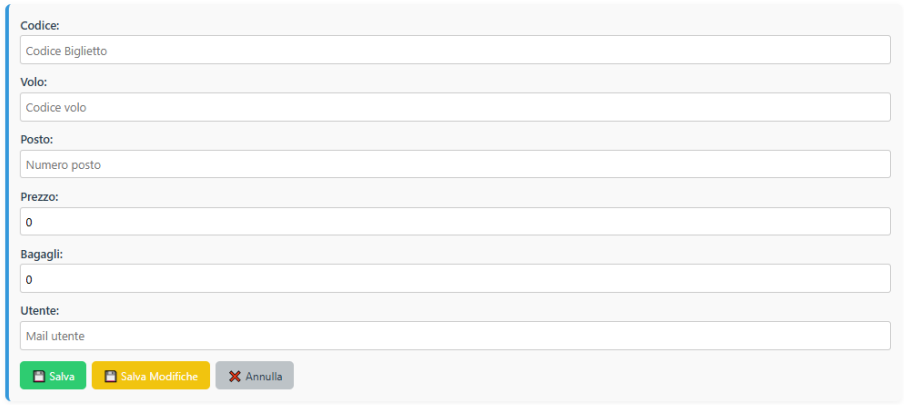
§ Aggiungere, modificare ed eliminare biglietti acquistabili. Quando vengono aggiunti nuovi biglietti è necessario che venga creato un biglietto per ogni posto libero presente nel velivolo;

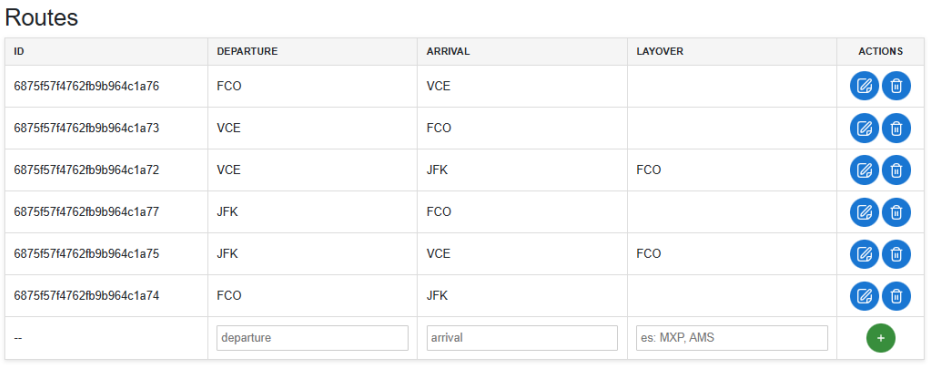
§ Aggiungere, modificare ed eliminare rotte.




Add Flight
Flight Code
Route ID
Aircraft Code
Duration (min)
gg/mm/aaaa --:--
Add
Flights List
Flight Code
Route
Aircraft
Duration
Date & Time
Actions
AB107
6875f57f4762fb9b964c1a75
4
640
2025-07-25 14:30
AB104
6875f57f4762fb9b964c1a72
1
640
2025-06-25 17:30


I tuoi Biglietti
Aggiungi Biglietto
Codice:
14
Volo:
AB109
Posto:
C1
Prezzo:
50
Bagagli:
3
Utente:
ITA.airways@project_flight.it
Salva
Salva Modifiche
Annulla

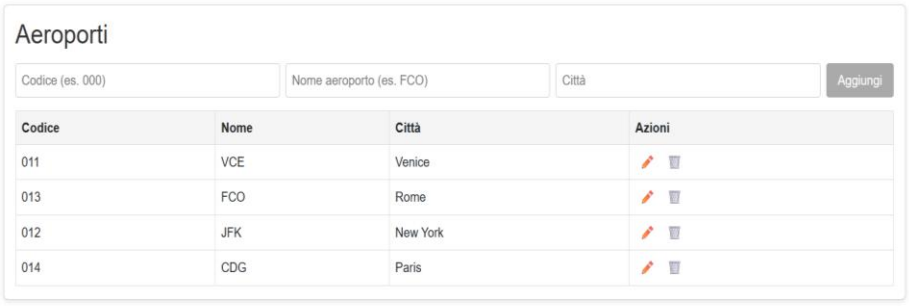


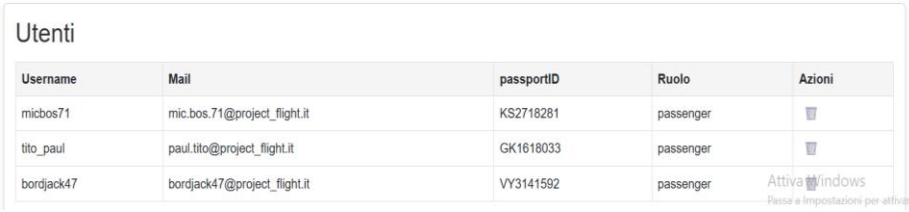


- Se l'utente è admin allora può:

§ Aggiungere, modificare ed eliminare aeroporti;

§ Visualizzare tutti gli utenti registrati come passeggeri ed eventualmente eliminarli





Bordignon Giacomo 897270

§ Invitare le compagnie aeree tramite un apposito form che permette di impostare i parametri necessari tra cui una password temporanea

---

